SHELFLINE — Master Build Plan (Phases and Steps)

Everything we will build, in order, from empty repo to demo day.

0. How to use this plan

- **Assumptions:** Python + FastAPI + Postgres backend, React frontend, 3–4 people. If you are fewer, follow the cut-line in Section 16 strictly.
- **Time budget:** percentages below are shares of your *total build hours*. The right-hand column shows hours for a 36-hour build window. Scale it to your real sprint length.
- **Order matters.** Phases 0–3 give you a thin, working, testable core. Phase 4 onward adds intelligence. Never start a later phase while an earlier one has failing tests.
- **Each phase ends with a 'Done when' test.** If it is not demonstrable, it is not done.
- **Every step is a commit** (or several). Message format: `feat(area): what changed`. Never batch a whole phase into one commit.

Phase	Name	Share	Hours (36 h)	Ships
0	Setup and foundations	5%	2	Repo, CI, docker, skeletons
1	Core primitives	9%	3.5	Clock, events, schema, ETA, Safety Gate
2	Entities, batches, ledger	7%	2.5	Batch intake, feasibility, shelter ledger
3	Lease engine	12%	4.5	Leases, reaper, progress detection
4	Planner	12%	4.5	Splitting, multi-drop, greedy EDF
5	Reroute and fallbacks	10%	3.5	Mid-transit recovery, tiers
6	Rules engine	5%	2	Compatibility, capabilities, vetoes
7	API and realtime	6%	2	Endpoints, WebSockets, roles

Phase	Name	Share	Hours (36 h)	Ships
8	Frontends	11%	4	Control Room, Rider PWA, Shelter, Donor
9	Simulator and Chaos Console	6%	2	Rider agents, scenarios, flood mode
10	Hardening	7%	2.5	Property, concurrency, restart tests
11	Stretch	4%	1.5	CP-SAT, reliability, racing
12	Docs, pitch, rehearsal	6%	2	README, video, Q&A prep
13	Freeze and demo protocol	(separate)	last 3 h	Frozen build, backups

Audit v2 — what would still make us fail

The first version was engine-strong but demo-weak: it would pass our own tests and still stumble in front of judges. We found 22 gaps; each is fixed below in bullets labelled 'Added in audit v2'.

ID	Gap	Why it would fail us	Fix	Where
G1	UI work starts late (frontends 'usable' at 70%)	Integration bugs surface in the last hours	API contract and mock server in Phase 0; frontend builds in parallel; walking skeleton by 35%	0.10, 10.12, checkpoints
G2	No deployment plan	Judges may open a URL on phones; geolocation, wake-lock and service workers need HTTPS	Smoke-deploy to a public HTTPS URL in the first hours; local fallback	0.11, 10.9, 13

ID	Gap	Why it would fail us	Fix	Where
G3	Real time vs demo time	A 75-minute window cannot be watched live	Judge Mode time scale (1 sim-minute is about 6 s)	1.9
G4	Judges act by hand, not through our simulator	A laptop has no GPS; 'unresponsive volunteer' means a judge doing nothing	GPS-less rider emulator, launcher with 5 personas, rider switchable to manual	8.11, 9.10
G5	Only planner-pushed offers; no self-claim	The handout's ghost-reservation story starts with a volunteer or NGO claiming	Self-claim and NGO-claim paths under the same lease rules	3.13, 3.14, 8.12
G6	No donor handover screen or actual pickup quantity	Split pickups cannot be demoed; shortfalls corrupt the ledger	Donor handover view; actual quantity adjusts lot and ledger	2.13, 8.10
G7	safe_until assumes cooked food	Medicines and kits use expiry dates and cold-chain budgets	Per-category safe_until strategies	1.13, 6.10
G8	Planner sees only idle riders and open shelters	Busy riders and shelter intake hours ignored	Projected free time and location; intake windows	2.12, 4.11
G9	Planner and events race each other	Stale plans overwrite fresh state; deadlocks	state_version check, lock ordering, one worker	3.16, 4.12
G10	Replay invariant is unrealistic	Re-running the planner or reading wall-clock breaks replay	Store plan outputs as events; seeded randomness	1.11, 10.10
G11	Clock jump skips intermediate deadlines	Fast-forward misorders expiries	advance_to steps through scheduled actions in order	1.10, 10.11

ID	Gap	Why it would fail us	Fix	Where
G12	Reliance on web push	Judge phones will not have granted permission	WebSocket, modal, sound, vibration, polling, SMS outbox	5.13
G13	Weak OTP and open admin endpoints	Guessable codes; anyone can hit reset on the public URL	Single-use OTP with attempt limits; admin key; rate limits	3.15, 7.8, 13
G14	No safety evidence beyond timers	Judges ask how you know it is safe	Optional temperature entry with cited thresholds	6.9
G15	Rider safety and dignity ignored	Countdown pressure can push riders to speed	Calm wording, no speed incentives, no shaming	8.13
G16	Units, duplicates, time zones	Trays vs portions; double-submit makes two batches	Conversion table, idempotency key from form, UTC storage	1.12, 2.10
G17	Rider cancels or refuses reroute after pickup	Cargo stuck with an unwilling rider	Refusal policy, handoff flow, ops alert	3.17, 5.12
G18	No judge-facing path	Judges do not know what to click; the four named failures are not one click each	Judge Mode, four presets, JUDGE_GUIDE.md	7.9, 8.14, 9.9, 12.7
G19	Git setup errors	Wrong author email means contributions may not count; secrets in history hurt evaluation	Verify author per member, .env.example, no squashed history	0.9, 12.8
G20	Scope: four apps in 4 hours	Frontend overruns and starves the engine	One React app with role routes; rebalanced budget	8.9, budget below

ID	Gap	Why it would fail us	Fix	Where
G21	External dependencies in the demo	Map tiles or routing APIs fail on venue wifi	Offline-first defaults; schematic map	0.12, 9.11
G22	Food not ready yet	Rider arrives before cooking is finished	ready_at constraint on offers and slack	2.11

Budget reality (honest). The 36-hour column means team wall-clock hours with people working in parallel, not person-hours; say so in your plan. Phase 8 is underbudgeted: four apps in 4 hours does not happen. Rebalance to 6 hours by taking 1.5 h from Phase 11 (stretch becomes leftover-only) and 0.5 h from Phase 2. Phase 3 at 4.5 h is tight, so build lease, expiry and reassign first and hot-standby racing last. If Phase 5 is not demoable at 50% of the time, cut Phase 11, Tier 3 flash offers and multi-language strings immediately.

Phase 0 — Setup and foundations

Goal: a fresh clone runs with one command and CI is green before any feature exists.

1. **0.1 Repo:** create the GitHub repo, add license, `.gitignore`, `README` stub with the problem statement and team names. Make the first commit within the first 15 minutes.
2. **0.2 Layout:** create the monorepo structure in Appendix A.
3. **0.3 Containers:** `docker-compose.yml` with Postgres, backend and frontend; named volume for data; healthchecks.
4. **0.4 Backend skeleton:** FastAPI app, settings via environment variables, structured JSON logging, `/health` endpoint, Alembic initialized.
5. **0.5 Frontend skeleton:** Vite + React + TypeScript, router with four empty routes (control, rider, shelter, donor), shared API client.
6. **0.6 CI:** GitHub Actions running ruff, pytest and a frontend build on every push; add the badge to the README.
7. **0.7 Conventions:** `Makefile` (`up`, `down`, `test`, `seed`, `demo`, `reset`), `docs/DECISIONS.md` template, commit message rule, short-lived branches or trunk with small commits (pick one and say so in the README).
8. **0.8 Roles and ownership:** assign phase owners (Appendix G) and agree on the interface contracts between modules before writing code.

Done when: `git clone` then `make up` shows a health check and an empty UI, and CI is green.

Added in audit v2:

- **0.9 (G19) Git hygiene:** every member sets the same email as their GitHub account and makes one commit in the first hour; add `.env.example`; never commit secrets or build artifacts; do not squash history away.
- **0.10 (G1) API contract first:** write the OpenAPI stub for the Appendix D endpoints and run a mock server so the frontend builds in parallel from hour 1.
- **0.11 (G2) Smoke deploy early:** deploy the skeleton to a public HTTPS URL within the first few hours (single VM or PaaS, one backend worker, reachable Postgres). Test on a real phone now, not at the end.
- **0.12 (G21) Offline-first defaults:** pre-pull Docker images, vendor frontend assets, and make every external service (map tiles, routing API) optional with a local fallback.

Phase 1 — Core primitives

Goal: the five building blocks everything else depends on.

1. **1.1 Clock:** interface `now()`; `RealClock` and `SimClock` (advance by N minutes, jump to time, speed multiplier). All code must call it; ban `datetime.now()` with a lint rule or grep test.
2. **1.2 Schema and migrations:** create the tables in Appendix B with Alembic migrations, indexes on `expires_at`, `deliver_by`, `status`, `shelter_id`.
3. **1.3 Event log:** `append_event(type, payload, actor, idempotency_key)`; unique idempotency key; a pure `apply(state, event)` function so state can be rebuilt by replay.
4. **1.4 Geo utilities:** haversine distance, bounding-box filter, simple coordinate validation.
5. **1.5 ETA service:** interface `eta(from, to, vehicle, now) -> Distribution`; v1 uses haversine × 1.35 circuity, speed profile by time-of-day, per-rider traffic multiplier override (so the Chaos Console can inject jams), noise to produce P50 and P90; add an in-memory cache with short TTL.
6. **1.6 Configuration tables:** shelf life per category, serve buffer, handover time, grace period, redispatch reserve, TTL min and max, stall thresholds. All values live in one config file with comments.

7. **1.7 Safety Gate:** `is_safe(lot, rider, shelter, now) -> (ok, reasons[])` combining deadline (P90), vehicle capacity and rule checks (rules plugged in during Phase 6). Reasons are an enum so the UI can explain refusals.
8. **1.8 Tests:** unit tests for clock, idempotency, ETA quantiles, gate decisions on boundary times (exactly at, one second before, one second after).

Done when: you can call `is_safe` on a hand-built scenario and get correct reasons, and replaying events reproduces identical state.

Added in audit v2:

- **1.9 (G3) Time-scale mode:** `Clock.speed` multiplies real time (Judge Mode default: 1 sim-minute is about 6 real seconds). Every UI countdown derives from server time and speed, so a 75-minute window can be watched in under 8 minutes.
- **1.10 (G11) Ordered time advance:** `advance_to(t)` executes every scheduled action in timestamp order, step by step, emitting events with sim timestamps. It never jumps over a deadline.
- **1.11 (G10) Determinism:** store planner results as `PLAN_COMMITTED` events and have replay apply them instead of recomputing; seed any randomness; no `datetime.now()` outside the Clock.
- **1.12 (G16) Time and units:** store UTC, display local; a unit conversion table (kg, trays, boxes to portions) used at intake.
- **1.13 (G7) Safe-until strategies:** `cooked_food` (cooked_at + shelf life), `dated_item` (expiry or best-before date), `cold_chain` (excursion budget in minutes out of the fridge). Each category declares its strategy in config.

Phase 2 — Entities, batches and the shelter ledger

Goal: accept supply honestly and track demand without double-counting.

1. **2.1 Entity CRUD:** donors, riders, shelters (seed-friendly), overflow sinks.
2. **2.2 Batch intake:** `POST /batches` validates quantity, category, cooked time and pickup window; computes `safe_until` and `deliver_by` using the formulas in the solution blueprint; stores diet and allergen tags.
3. **2.3 Lots:** split a batch into lots (default lot size = largest vehicle capacity, adjustable); lot statuses `UNASSIGNED, LEASED, IN_TRANSIT, DELIVERED, EXPIRED, DISCARDED, ABORTED_UNSAFE`.

4. **2.4 Feasibility on creation:** quick reachability check (which riders can reach the donor, which shelters are reachable by `deliver_by`) returning `ACCEPTED_FULL`, `ACCEPTED_PARTIAL(n)` or `REJECTED_INFEASIBLE` with reason and the nearest viable alternative.
5. **2.5 Batch revision:** donor can reduce quantity or withdraw; lots rebalanced; affected leases released cleanly.
6. **2.6 Shelter ledger:** `reserve(lot, qty)`, `release`, `confirm` implemented as atomic conditional updates; `need`, `received`, `committed` columns; `available = need - received - committed`.
7. **2.7 Shelter operations:** `demand_changed`, `walk_in_received(+n)`, `close`, `reopen`; each emits an event and a re-evaluation trigger (wired in Phase 5).
8. **2.8 Expiry sweep:** lots past `deliver_by` become `EXPIRED` automatically (scheduled action).
9. **2.9 Tests:** ledger never goes negative; 50 concurrent reserves against 50 available portions yield exactly 50 reserved; infeasible batch returns the correct response (S01).

Done when: S01 and S11 pass and the concurrency test on the ledger passes.

Added in audit v2:

- **2.10 (G16) Intake safety:** convert units to portions on creation; the donor form sends an idempotency key so a double-tap creates one batch.
- **2.11 (G22) Ready-at constraint:** a lot has `ready_at`; it cannot be offered or leased before it, and slack counts from `ready_at`.
- **2.12 (G8) Projected availability:** riders carry `free_at` and `free_location` (end of the current job); shelters carry intake windows (opening and closing time).
- **2.13 (G6) Actual quantity at pickup:** the donor reports what was really handed over; a shortfall shrinks the lot, adjusts the ledger and triggers re-planning for the remainder.

Phase 3 — Lease engine (the anti-ghost core)

Goal: no reservation can outlive its usefulness.

1. **3.1 State machine:** table of allowed transitions (Appendix E); any other transition raises an error and is tested.
2. **3.2 TTL calculation:** `lease_ttl = clamp(P90(rider→donor) + grace, min, latest_pickup - now - redispatch_reserve)`.

3. **3.3 Offer, accept, decline:** offer expires quickly (for example 60 s) if not accepted; accept creates the soft lease and reserves ledger capacity atomically.
4. **3.4 Progress ingestion:** location pings and milestone taps (`ARRIVED_DONOR` , `PICKED_UP` , `NEAR_SHELTER`); progress detector computes distance-to-target trend; three non-shrinking pings mark `STALLED` .
5. **3.5 Lease extension rule:** progress may extend the soft lease, never beyond `latest_pickup - redispatch_reserve` .
6. **3.6 Pickup commit:** donor-side OTP or QR confirmation moves the lease to hard commit and the lot to `IN_TRANSIT` .
7. **3.7 Delivery confirm:** shelter confirms received quantity (may differ from planned); ledger moves committed → received; leftover quantity triggers follow-up routing.
8. **3.8 Reaper and scheduled actions:** persisted `scheduled_actions` (`NUDGE` , `ALERT_STANDBY` , `EXPIRE` , `SMS_FALLBACK`) executed by a loop every 2–5 s inside transactions with `FOR UPDATE SKIP LOCKED` .
9. **3.9 Escalation ladder:** nudge → standby alert → expiry and reassignment, each step an event and a notification outbox entry.
10. **3.10 Hot standby:** shadow offer to the next-best rider when slack is thin; first pickup confirmation wins; loser auto-released.
11. **3.11 Ghost return handling:** expired rider's later actions get a polite, deterministic rejection with an alternative offer.
12. **3.12 Tests:** S03, S04, S08, S20, S21 as automated scenarios using `SimClock` .

Done when: you can start a lease, stop sending pings, fast-forward the sim clock, and watch the lease expire and the lot get reassigned with no human action.

Added in audit v2:

- **3.13 (G5) Self-claim:** `GET /lots/nearby` returns only lots passing the Safety Gate for this rider, with refusal reasons for the rest; `POST /lots/{id}/claim` creates a soft lease under the same TTL, ledger and expiry rules as planner offers. This also gives a walking skeleton that works before the planner exists.
- **3.14 (G5) NGO claim:** an NGO coordinator can claim for the organization, then must assign a named rider within the lease TTL or the claim lapses and the lot returns to the pool. This reproduces the handout's ghost-reservation story and shows it being defeated.
- **3.15 (G13) OTP security:** pickup and delivery codes are single-use, bound to one lease, expire with it, and lock after 5 wrong attempts.

- **3.16 (G9) Concurrency discipline:** every state change checks `state_version`; locks are always taken in the same order (shelter, then lot, then lease, by id); the demo runs one backend worker with one reaper and one planner.
- **3.17 (G17) Rider cancels after pickup:** the cargo is already out, so cancellation triggers a handoff to the nearest standby rider or handoff point with a new OTP, and an ops alert if nobody accepts within 60 s.

Phase 4 — Planner

Goal: turn supply, riders and demand into feasible assignments, fast.

1. **4.1 Candidate generation:** for each unassigned lot, list (rider, shelter) pairs passing the Safety Gate with P90 times.
2. **4.2 Scoring:** `value = safe_portions × urgency(shelter) - α·detour - β·P(late) - γ·(1 - reliability)`; weights in config.
3. **4.3 Greedy EDF:** process lots by least slack first; assign best candidate; update ledger holds in memory during the pass.
4. **4.4 Multi-drop builder:** insertion heuristic to add a second or third stop to a rider's route, checking P90 at each stop against `deliver_by` and vehicle capacity.
5. **4.5 Splitting:** divide a lot across riders when no single rider can carry it; divide across shelters when no shelter can absorb it.
6. **4.6 Plan object:** persist plans with version numbers; compute diff against the current leases; apply changes with hysteresis (skip if gain below threshold).
7. **4.7 Triggers:** run on `BATCH_CREATED`, `RIDER_AVAILABLE`, `LEASE_EXPIRED`, `DEMAND_CHANGED`, plus a 30 s tick; debounce bursts; hard time box of 200 ms for the greedy pass.
8. **4.8 Surplus handling:** unplaceable portions become open offers (Tier 3 pre-stage) or are flagged infeasible with reason.
9. **4.9 Explainability:** every assignment stores a human-readable reason (for example 'chosen: slack 14 min, reliability 0.9; rejected R4: P90 arrival after deliver_by').
10. **4.10 Tests:** S02 (180 meals, capacity 40, shelter need 50) produces a valid split plan; no plan violates the Safety Gate; planner runtime benchmarks recorded.

Done when: S02 works end to end through the API and the plan explanation is visible.

Added in audit v2:

- **4.11 (G8) Projected availability:** candidates include riders who will be free by `free_at` at `free_location`, using P90 travel from there; idle-only planning wastes capacity during a flood.
- **4.12 (G9) Optimistic plan application:** a plan records the `state_version` it was computed on; if the version changed before apply, discard and recompute rather than overwrite fresh state.

Phase 5 — Rerouting and fallbacks

Goal: cargo in motion is never stranded.

1. **5.1 Fallback computation:** for each in-transit lease, compute and store the top 3 alternative destinations using current GPS, P90 ETA, ledger availability and rules; refresh every 30 s and on any relevant event.
2. **5.2 Event handlers:** `DEMAND_CHANGED`, `SHELTER_CLOSED`, `WALK_IN_RECEIVED`, `CAPABILITY_CHANGED`, `TRAFFIC_SPIKE`, `RIDER_BREAKDOWN`, each finding affected leases.
3. **5.3 Reroute decision:** choose from stored fallbacks after re-validation; emit `REROUTE_PROPOSED`.
4. **5.4 Partial drop and in-flight split:** drop what the current shelter can still take, continue with the remainder.
5. **5.5 Tier 2 sinks:** registry of overflow destinations with open hours and intake limits.
6. **5.6 Tier 3 flash offer:** broadcast to eligible shelters and NGOs, 10-minute soft lease, only when slack permits.
7. **5.7 Tier 4 safe discard:** documented reason, counted in the waste metric, requires explicit confirmation from rider or shelter.
8. **5.8 Rider acknowledgment:** one-tap accept; auto-accept option; 60 s timeout escalates to SMS then call; timeouts logged.
9. **5.9 Breakdown handling:** nearest standby rider or handoff point; cargo handoff flow and re-OTP.
10. **5.10 Metrics:** decision time and acknowledgment time per reroute.
11. **5.11 Tests:** S05, S06, S07, S09, S12, S15.

Done when: closing a shelter while a lease is IN_TRANSIT produces a new valid route in under one second of server time, and the demo shows which tier was used.

Added in audit v2:

- **5.12 (G17) Refusal policy:** if a rider declines a reroute, they must still serve the original shelter if it can take the load; otherwise the tier flow continues with a handoff and an ops alert. Every refusal is logged and lowers reliability only mildly.
- **5.13 (G12) Reliable alerts:** deliver reroutes by WebSocket with a full-screen modal, sound and vibration, with polling as fallback and an SMS outbox entry. Do not depend on web push, which judges' phones will not have enabled.

Phase 6 — Rules engine and safety vetoes

Goal: compatibility and dependency rules are data, not code.

1. **6.1 YAML schema:** items, `requires`, `rider_requires`, `max_ambient_minutes`, `diet_must_match`; loader with validation and clear errors.
2. **6.2 Capability model:** shelter capabilities (water, fuel, fridge, serving capacity) with `verified_at` and TTL; stale means `UNKNOWN`.
3. **6.3 Reconfirm flow:** request one-tap reconfirmation from the shelter when a needed capability is `UNKNOWN`.
4. **6.4 Diet and allergen tags:** match batch tags to shelter acceptance preferences.
5. **6.5 Vehicle rules:** volume and insulated box requirements.
6. **6.6 Gate integration:** rules evaluated inside `is_safe` and the candidate filter; re-evaluated in every replan and on `CAPABILITY_CHANGED`.
7. **6.7 Safety veto:** `report_unsafe(lot, reporter, reason)` stops the lot, marks `ABORTED_UNSAFE`, blocks redistribution, and logs the audit trail.
8. **6.8 Tests:** S13, S14, S17, S18.

Done when: flipping a shelter's water capability to false during the demo immediately removes ration kits from its candidates and reroutes any in-transit kit lease.

Added in audit v2:

- **6.9 (G14) Safety evidence:** optional temperature entry at pickup and delivery, with thresholds taken from the guideline you cite in the README; an out-of-range value blocks auto-confirm and flags the lot.
- **6.10 (G7) Medicines behind a flag:** off by default; when on, require a pharmacist-verified consignee and block prescription items. State the legal limits in the pitch rather than claiming compliance.

Phase 7 — API and realtime layer

Goal: clean, documented interfaces for all four apps and the simulator.

1. **7.1 Endpoints:** implement the list in Appendix D, with request and response models and OpenAPI docs.
2. **7.2 Idempotency:** `Idempotency-Key` header on all mutating endpoints.
3. **7.3 Roles:** simple role tokens (donor, rider, shelter, admin) issued via short codes; enforce per-endpoint.
4. **7.4 WebSockets:** channels `control`, `rider/{id}`, `shelter/{id}`; events pushed as they are appended; reconnect with last-seen event id to replay missed events.
5. **7.5 Snapshot endpoint:** `GET /state/snapshot` for initial load and for debugging.
6. **7.6 Rate limits and validation:** pydantic strictness, reasonable limits to protect against double-tap floods.
7. **7.7 Error model:** machine-readable error codes (`LEASE_EXPIRED`, `LOT_NOT_SAFE`, `SHELTER_FULL`, and so on) so the UI can show useful messages.

Done when: a script can run the full S02 scenario using only HTTP calls and receive correct WebSocket updates.

Added in audit v2:

- **7.8 (G13) Admin protection:** chaos, clock, reset and seed endpoints require an admin key and are rate-limited; judges get a Judge role that can run presets but cannot wipe data.
- **7.9 (G18) Preset endpoints:** `POST /admin/presets/{name}` for the four named judge failures (see 9.9) so each runs in one click.

Phase 8 — Frontends

Goal: four small, clear apps that make the engine visible.

1. **8.1 Design system:** status colors (green safe, amber at risk, red critical), countdown component, slack bar component, consistent typography.
2. **8.2 Control Room:** map with riders, donors and shelters; batch table with live slack bars; lease timeline; event feed; metrics strip (portions saved, wasted, violations, ghost rate); plan explanation drawer.
3. **8.3 Map fallback:** if map tiles are unreachable (venue wifi), fall back to a schematic SVG map so the demo does not depend on the internet.

4. **8.4 Rider PWA:** join via code, availability toggle, job card with countdown, big buttons (accept, arrived, picked up with OTP, near shelter, delivered), reroute modal with one-tap accept, offline queue for events, screen wake-lock during active job, basic Hindi and English strings.
5. **8.5 Shelter tablet:** need and received counters, '+ received outside' button, close or reduce demand, capability toggles with 'last verified' time, confirm delivery quantity, report unsafe.
6. **8.6 Donor form:** category presets with default shelf life, cooked-at picker, quantity, tags; shows the feasibility response and reasons immediately.
7. **8.7 SMS outbox panel:** shows nudges and fallback messages so judges see the non-app path.
8. **8.8 Polish pass:** mobile responsive, loading and error states, keyboard accessible, no console errors.

Done when: a non-team member can create a batch on the donor form and watch it flow through the Control Room without help.

Added in audit v2:

- **8.9 (G20) One app, not four:** a single React app with role routes sharing components and the API client; the Rider view starts as a minimal mobile page, not a full PWA.
- **8.10 (G6) Donor handover view:** lists the lots of the batch, which riders are arriving, OTP entry, actual quantity handed over, and a progress line such as 2 of 5 lots collected. Without it, split pickups cannot be demonstrated.
- **8.11 (G4) Rider emulator:** for laptops without GPS, a draggable marker plus a 'drive toward target' button at a chosen speed; a Rider launcher opens up to 5 rider personas in separate tabs. A judge simulates an unresponsive volunteer simply by doing nothing in a tab.
- **8.12 (G5) Nearby lots list:** shows claimable lots, with the Safety Gate's reasons for greyed-out ones.
- **8.13 (G15) Rider-safe wording:** no shaming, a calm timer, a standing 'ride at a safe speed' line, and no incentives tied to speed.
- **8.14 (G18) Judge Mode panel:** a banner, the time-scale selector, the four failure presets and a link to the judge guide.

Phase 9 — Simulator and Chaos Console

Goal: judges can break the system on purpose, and you can prove recovery.

1. **9.1 Rider agents:** behaviors `RELIABLE`, `SLOW`, `FLAKY`, `GHOST`, `DIES_AFTER_PICKUP`; move along straight-line paths at ETA-derived speed; emit pings and milestones.
2. **9.2 Seed data:** a realistic city layout with around 10 donors, 8 shelters, 15 riders, 3 overflow sinks; one command seeds it.
3. **9.3 Chaos endpoints:** `ghost_rider`, `traffic_multiplier`, `close_shelter`, `reduce_need`, `walk_in`, `kill_connection`, `donor_reduces_qty`, `double_claim`, `advance_clock`, `flip_capability`, `rider_breakdown`; each logs `CHAOS_INJECTED`.
4. **9.4 Chaos Console UI:** one button per endpoint, parameter pickers, last-action log.
5. **9.5 Scenario scripts:** S01–S26 (Appendix H) as JSON files runnable through the simulator, with expected outcomes asserted.
6. **9.6 Flood mode:** generator that injects 30 batches and 15 riders within a minute to test planner load.
7. **9.7 Scoreboard:** end-of-run summary (delivered, wasted by cause, violations, reroute times).
8. **9.8 Reset:** `make reset` returns the demo to a clean seeded state in under 10 seconds.

Done when: every scenario in the catalog can be triggered from the console or from `make demo SCENARIO=S06`.

Added in audit v2:

- **9.9 (G18) Four judge presets, one click each:** (a) a batch with a tight expiry window, (b) one batch split across several riders with partial pickups, (c) an unresponsive volunteer after claiming, (d) a shelter's demand closing while a delivery is in transit. Each shows its expected outcome beside the live result.
- **9.10 (G4) Manual rider switch:** any simulated rider can be flipped to manual so a judge drives it from a tab.
- **9.11 (G21) Offline mode:** a flag that disables every external call and uses the schematic map.

Phase 10 — Hardening

Goal: find your own bugs before the judges do.

1. **10.1 Property tests** (Hypothesis) for the invariants in Appendix F over random event sequences.
2. **10.2 Concurrency tests:** 50 simultaneous claims, simultaneous demand change and reserve, duplicate requests with same idempotency key.

3. **10.3 Restart test:** kill the backend mid-scenario; on restart the reaper resumes and no lease is lost or duplicated.
4. **10.4 Time-boundary tests:** actions exactly at `deliver_by`, at lease expiry, and at slack equals zero.
5. **10.5 Load test:** flood mode with planner time box respected; record P95 planner time.
6. **10.6 Bug bash:** someone not on the team tries to break the system for 30 minutes using the Chaos Console; file and fix findings.
7. **10.7 Security basics:** input validation, role checks on every endpoint, no secrets in the repo, rate limiting.
8. **10.8 Observability:** request ids in logs, event feed searchable, `/metrics` endpoint.

Done when: CI runs the property and concurrency suites green three times in a row.

Added in audit v2:

- **10.9 (G2) Real-device test:** run the deployed URL on Android Chrome and iPhone Safari over mobile data; geolocation, wake-lock and background-tab behavior differ, so confirm each fallback works.
- **10.10 (G10) Replay test:** rebuild state from the event log and compare it with live state.
- **10.11 (G11) Clock-jump test:** `advance_to(+40 min)` must produce the same outcome as 40 one-minute steps.
- **10.12 (G1) Integration smoke test:** CI runs the skeleton flow against the deployed staging URL on every merge to main.

Phase 11 — Stretch (only after Phase 10 is green)

Do these in this order; stop when time runs out.

1. **11.1 Reliability score:** Beta-distribution estimate per rider updated on completions and expiries; feeds TTL length and scoring.
2. **11.2 Hot-standby racing polish:** compensation credit for the loser, clear UX.
3. **11.3 CP-SAT improver:** OR-Tools model over the candidate set with a 1.5 s limit; accept only if it beats greedy by a margin; fall back to greedy on timeout.
4. **11.4 Fairness weighting:** waiting-time boost for distant or repeatedly skipped shelters.
5. **11.5 Companion dispatch:** create a linked water or fuel task when a shelter's missing capability blocks a kit delivery.
6. **11.6 Analytics page:** time-series of waste, slack at delivery, ghost rate.

7. **11.7 Messaging mock:** WhatsApp-style rider notifications alongside SMS.

Phase 12 — Documentation, pitch and rehearsal

1. **12.1 README:** problem in three lines, architecture diagram (Mermaid), design laws, one-command setup, 'break it yourself' section, honest limitations.
2. **12.2 Docs:** `docs/ARCHITECTURE.md`, `docs/SCENARIOS.md` (S01–S26 with expected behavior and test names), `docs/DECISIONS.md`.
3. **12.3 Demo video:** 2–3 minutes, screen-recorded Chaos Console run, as a backup if live demo fails.
4. **12.4 Pitch:** five slides (problem, insight, architecture, live demo, results) and the three-sentence pitch.
5. **12.5 Judge Q&A sheet:** prepare answers for: Why P90? What if GPS is wrong? How do you handle fraud? What is simulated versus real? How would this scale beyond one city? What are the legal limits for medicines?
6. **12.6 Rehearsal:** three full run-throughs with a timer; each person can run the demo alone.

Added in audit v2:

- **12.7 (G18) JUDGE_GUIDE.md:** one page with four recipes (each under 2 minutes) for the failures judges named, plus the URL, seed accounts, the expected result and what to do if something looks wrong.
- **12.8 (G19) Repo audit before submission:** each member's commits show under their own GitHub profile, no secrets in history, CI badge green, README setup works on a fresh machine.

Phase 13 — Freeze and demo protocol

1. **T-3 h: code freeze.** Only fixes for demo-blocking bugs, each with a test.
2. **Fresh-machine test:** clone on a different laptop and run `make up && make seed`.
3. **Backups:** recorded video, screenshots of each scenario, a local copy of the repo on a USB drive, hotspot as network backup.
4. **Demo checklist:** reset, seed, sim clock at 10x, Chaos Console open, SMS outbox visible, phone with Rider PWA logged in.
5. **Final commits:** tag `v1.0-demo`, ensure README matches reality, push before presenting.

Added in audit v2 (G2, G13):

- Verify the deployed URL on the judges' likely devices, show a QR code for it, keep a hotspot and the local build as fallback, and confirm the admin and reset endpoints are protected before the doors open.

14. Dependency order (what blocks what)

`Phase 0 → 1 → 2 → 3 → 4 → 5` is the critical chain. Phases 6 and 7 can run in parallel once Phase 3 is done. Phase 8 starts in the first hours against the mocked API contract from Phase 0 and integrates continuously; it does not wait for Phase 7. Phase 9 can start as soon as Phase 3 exists. Phase 10 runs continuously but gets a dedicated block at the end.

15. Suggested team split

Role	Owns	Also does
Engine lead	Phases 1, 3, 4, 5	Code review of safety-critical paths
Backend and rules	Phases 2, 6, 7	Scenario tests S01, S13, S14
Frontend	Phase 8	Chaos Console UI
Sim, QA and docs	Phases 9, 10, 12	Bug bash, demo video, pitch

If you are three, merge frontend and sim. If you are two, merge engine with backend and frontend with sim/docs.

16. Cut-line (decide now, not at 3 a.m.)

- **Never cut:** Phase 0–5 core behavior, Safety Gate, lease reaper, ledger atomicity, Chaos Console basics, tests for S02, S03, S06, S08.
- **Cut first:** Phase 11 entirely, then multi-language strings, then Tier 3 flash offers, then the analytics page.
- **Cut second:** schematic map polish, WhatsApp mock, companion dispatch.
- **Cut last:** hot-standby racing and the rules YAML (keep a hard-coded minimal version if forced).

- **Rule:** if a phase is more than 25% over budget, stop and cut within that phase rather than borrowing from hardening.

Appendix A — Repository layout

```
shelfline/
  backend/
    app/
      main.py
      config.py
      clock.py
      events.py
      db/ (models.py, migrations/)
      geo.py
      eta.py
      safety_gate.py
      rules/ (engine.py, rules.yaml)
      ledger.py
      batches.py
      leases.py
      reaper.py
      planner/ (candidates.py, scoring.py, greedy.py, multidrop.py,
solver.py)
      reroute.py
      notifier.py
      api/ (routes_*.py, ws.py, auth.py)
      sim/ (agents.py, chaos.py, scenarios/, seed.py)
    tests/ (unit/, scenarios/, property/, concurrency/)
  frontend/
    src/ (control/, rider/, shelter/, donor/, shared/)
  docs/ (ARCHITECTURE.md, SCENARIOS.md, DECISIONS.md)
  docker-compose.yml
  Makefile
  README.md
  .github/workflows/ci.yml
```

Appendix B — Database tables

Table	Purpose
donors, riders, shelters	Actors with location, status and contact
shelter_capabilities	Capability name, value, verified_at
overflow_sinks	Tier 2 destinations with hours and limits
batches	Donor supply with timing and tags
lots	Splittable slices with status
leases, lease_stops	Rider commitments and their ordered drops
plans	Versioned planner outputs and explanations
events	Append-only log with idempotency key
scheduled_actions	Persisted timers executed by the reaper
notifications_outbox	Push and SMS messages, status
metrics_snapshots	Periodic aggregates for the scoreboard

Appendix C — Event types

`BATCH_CREATED, BATCH_REVISED, LOT_CREATED, LEASE_OFFERED, LEASE_ACCEPTED, LEASE_DECLINED, RIDER_PING, RIDER_MILESTONE, LEASE_STALLED, LEASE_EXPIRED, LEASE_RELEASED, PICKUP_CONFIRMED, DELIVERY_CONFIRMED, DEMAND_CHANGED, SHELTER_CLOSED, WALK_IN_RECEIVED, CAPABILITY_CHANGED, REROUTE_PROPOSED, REROUTE_ACKED, UNSAFE_REPORTED, LOT_EXPIRED, LOT_DISCARDED, PLAN_COMMITTED, CHAOS_INJECTED`

Appendix D — API surface

Group	Endpoints
Batches	`POST /batches`, `GET /batches/{id}`, `PATCH /batches/{id}`

Group	Endpoints
Leases	`POST /leases/{id}/accept`, `/decline`, `/progress`, `/milestone`, `/pickup`, `/deliver`
Riders	`POST /riders/{id}/availability`, `/location`, `/breakdown`
Shelters	`POST /shelters/{id}/demand`, `/walk-in`, `/close`, `/reopen`, `/capabilities`, `/report-unsafe`
Admin and chaos	`POST /admin/clock`, `/admin/chaos/{type}`, `/admin/reset`, `/admin/seed`
Read	`GET /state/snapshot`, `GET /metrics`, `GET /events?since=`
Realtime	`WS /ws/control`, `/ws/rider/{id}`, `/ws/shelter/{id}`

Appendix E — Lease state transitions

From	To	Trigger
OFFERED	ACCEPTED	Rider accepts within offer window
OFFERED	EXPIRED	Offer window passes
ACCEPTED	ARRIVED_DONOR	Milestone or geofence
ACCEPTED	EXPIRED	TTL passes without sufficient progress
ACCEPTED	RELEASED	Rider cancels or planner reassigns
ARRIVED_DONOR	PICKED_UP	OTP confirmed (hard commit)
PICKED_UP	NEAR_SHELTER	Milestone or geofence
PICKED_UP	REROUTED	Reroute accepted
NEAR_SHELTER	DELIVERED	Shelter confirms
Any active	ABORTED_UNSAFE	Safety veto

Appendix F — Invariants (property-tested)

1. No delivery completes after `deliver_by` without being flagged as a safety exception.
2. For every shelter, `received + committed <= need` (unless soft overbook is explicitly on).
3. A lot never has more than one hard-committed lease.
4. Sum of lease quantities for a lot never exceeds the lot quantity.
5. No new lease is created for an expired, discarded or unsafe lot.
6. Every in-transit lease has at least one valid fallback or an explicit 'none available' marker.
7. Replaying the event log reproduces the current state exactly.
8. A rider's carried quantity never exceeds vehicle capacity.

Appendix G — Weekly-style checkpoints (use as a timeline)

Checkpoint	Must be true
15% of time	Phases 0–1 done, API contract and mocks published, staging deployed, first commits pushed
35%	Phases 2–3 done, ghost-lease scenario passes, walking skeleton (batch, self-claim, delivered) works through the UI
55%	Phases 4–5 done, S02 and S06 demo end to end
70%	Phases 6–7 done, frontends already integrated and demoable on the live API
85%	Phases 8–9 done, Chaos Console live
95%	Phase 10 done, stretch only if green
Final 3 h	Phase 12–13: docs, rehearsal, freeze

Appendix H — Scenario catalog (S01–S26)

ID	Situation	Required behavior
S01	Batch window shorter than any rider can meet	Immediate infeasibility response with reason and nearest viable option
S02	180 meals vs rider capacity 40 and shelter need 50	Auto-split into lots, multi-rider, multi-drop plan
S03	Rider claims then goes silent	Nudge, standby alert, expiry, reassign, ghost rider notified
S04	Rider alive but stuck in traffic	STALLED flag, ETA recalculated, standby promoted if P90 breaches
S05	Shelter fills by walk-in donation mid-route	Ledger update and instant reroute from precomputed fallback
S06	Shelter closes mid-transit	Tiered fallback, partial drops
S07	Shelter cuts need from 50 to 20 with 40 in transit	Drop 20, continue 20 to the next best
S08	Two riders claim the same lot simultaneously	Atomic winner; loser gets an alternative
S09	Rider phone dies after pickup	Last known position, shelter alerted, ops escalation, no reclaim
S10	Donor revises quantity down	Lots rebalanced, excess leases released
S11	Batch passes safe-until unassigned	EXPIRED, no new leases, counted as waste
S12	ETA breaches deliver_by after a traffic spike	AT_RISK, reroute or handoff
S13	Shelter lacks water for raw ration kits	Kits filtered out; alternative shelter or companion task
S14	Shelter capability data is stale	Treated as UNKNOWN; one-tap reconfirm
S15	Rider breaks down mid-route	Nearest standby or handoff point; reassign if slack allows
S16	Flood of simultaneous events	Time-boxed planner, greedy fallback

ID	Situation	Required behavior
S17	Donor misreports cooking time	Conservative defaults; rejection on arrival is allowed
S18	Shelter refuses delivery as unsafe	ABORTED_UNSAFE, no redistribution of that lot
S19	Rider loses connectivity	Offline queue, SMS fallback
S20	Server restarts mid-operation	Events and persisted deadlines restore state; reaper resumes
S21	Same command sent twice	Idempotency key returns the original result
S22	Delivery attempted after deadline	No auto-confirm; flagged safety exception
S23	NGO claims a lot but never assigns a rider	Claim lapses at TTL; lot returns to the pool
S24	Donor hands over fewer portions than planned	Lot shrinks, ledger adjusts, remainder re-planned
S25	Rider refuses a reroute	Serve original shelter or hand off; ops alert
S26	Clock fast-forward of 40 minutes	Deadlines processed in order; same result as stepping